

DATA - 260
HomeWork - 3
Name: Karthik Pragada
SJSU ID: 018768360

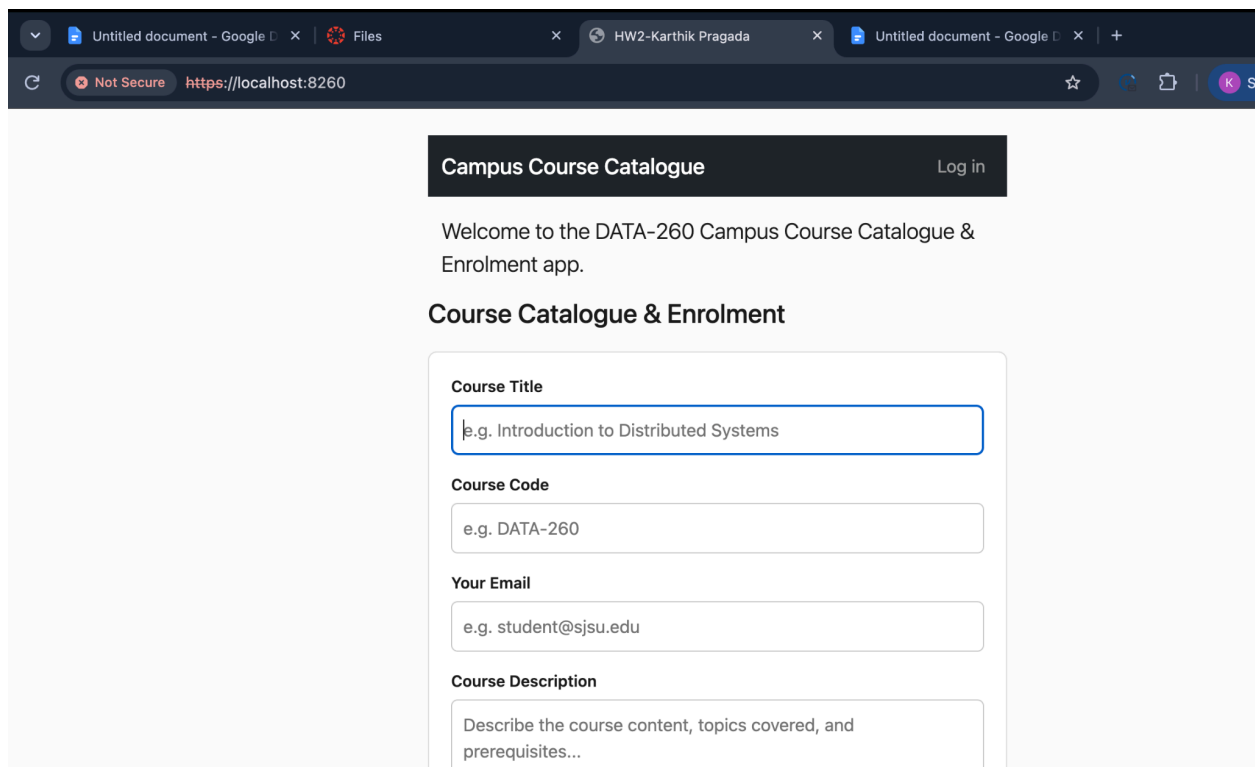
SID	8360
PORT_BASE	8260
PREFIX	s8360
SEED	8360
VERIFY_SEED	268360
DOMAIN_ID	0 (Campus Course Catalogue and Enrolment)
HARDWARE	MacBook Air, Apple M4, 16 GB RAM
Local Model Used	sentence-transformers/all-MiniLM-L6-v2 (HuggingFace, run locally) for Part 2
Tagged commit hash	1428376a034c26cf1d37339e06b11521e624097 b
Git repo	https://github.com/prags8585/data260-8360/tree/ main

Part 1 (FastAPI Authentication)

Added Bootstrap-styled login, logout and session management to the existing HW1/2 Course Catalogue app, extending main.py in place and adding a new auth.py router. Sessions use Starlette's SessionMiddleware with a Secure, HttpOnly, SameSite=Lax cookie, and the app uses one hard-coded demo user (username karthik, password data260) instead of a user database. The app was run over a self-signed local HTTPS certificate for the security tests, so the Secure flag is actually enforced.

1. Home Page (/) <https://localhost:8260/>

Home Page is a welcome banner with a "Log in" link in the navbar. The existing Course Catalogue form/list from previous Hw's remains under it, unchanged.



The screenshot shows a web browser window with the address bar displaying "https://localhost:8260". The page has a dark header with the text "Campus Course Catalogue" and a "Log in" link. Below the header, there is a welcome message: "Welcome to the DATA-260 Campus Course Catalogue & Enrolment app." followed by the section title "Course Catalogue & Enrolment". The main content area contains a form with four input fields: "Course Title" (with placeholder text "e.g. Introduction to Distributed Systems"), "Course Code" (with placeholder text "e.g. DATA-260"), "Your Email" (with placeholder text "e.g. student@sjsu.edu"), and "Course Description" (with placeholder text "Describe the course content, topics covered, and prerequisites...").

Campus Course Catalogue

Log in

Welcome to the DATA-260 Campus Course Catalogue & Enrolment app.

Course Catalogue & Enrolment

Course Title

e.g. Introduction to Distributed Systems

Course Code

e.g. DATA-260

Your Email

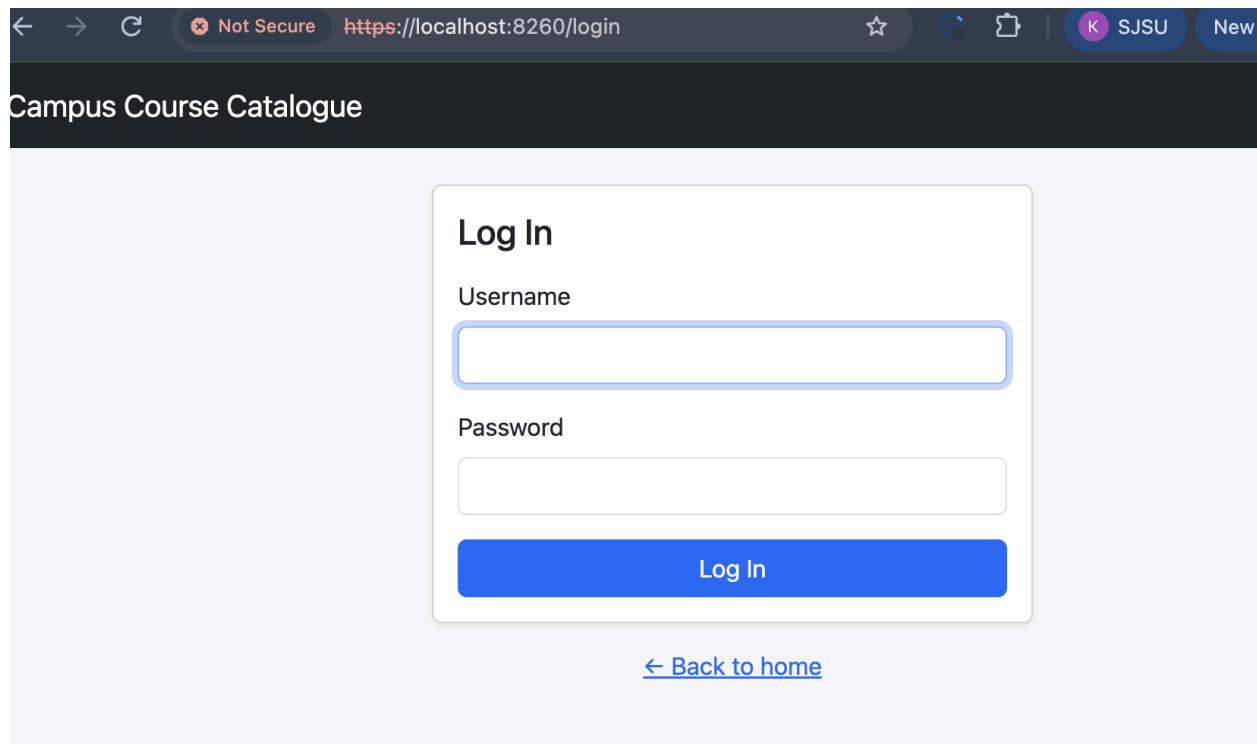
e.g. student@sjsu.edu

Course Description

Describe the course content, topics covered, and prerequisites...

2) Login Page (/login) : <https://localhost:8260/login>

When clicked on login we can see the below login page.



Not Secure <https://localhost:8260/login> SJSU New Chrome

Campus Course Catalogue

Log In

Username

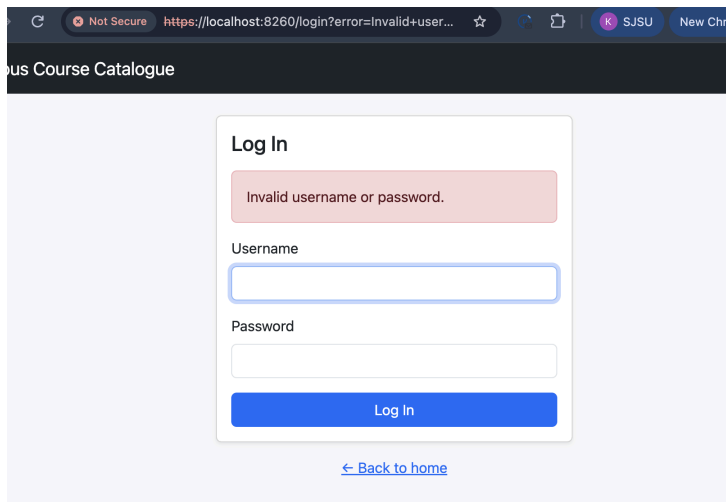
Password

Log In

[< Back to home](#)

a) Invalid credentials

An alert-danger is shown and the form is re-rendered if the user ID and Password don't match.



Not Secure <https://localhost:8260/login?error=Invalid+user...> SJSU New Chrome

Campus Course Catalogue

Log In

Invalid username or password.

Username

Password

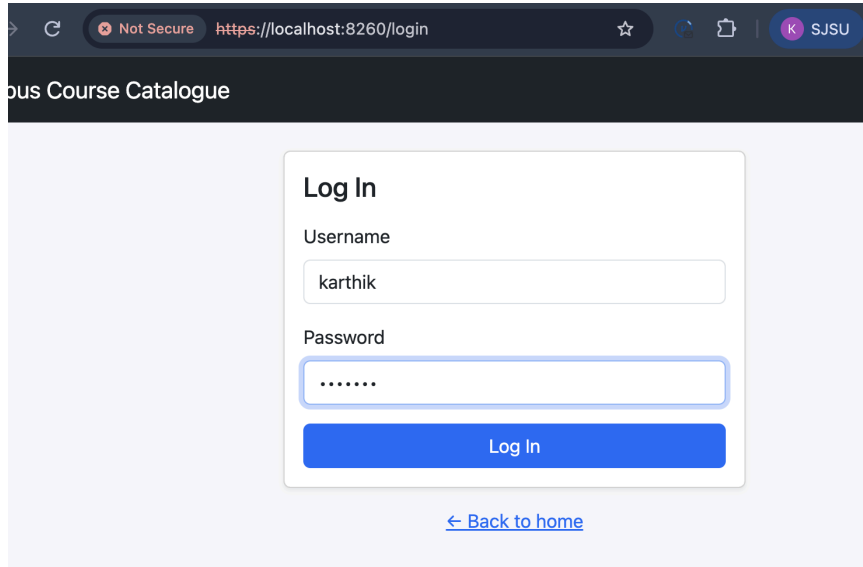
Log In

[< Back to home](#)

3) Dashboard Page (/dashboard) - protected route <https://localhost:8260/dashboard>

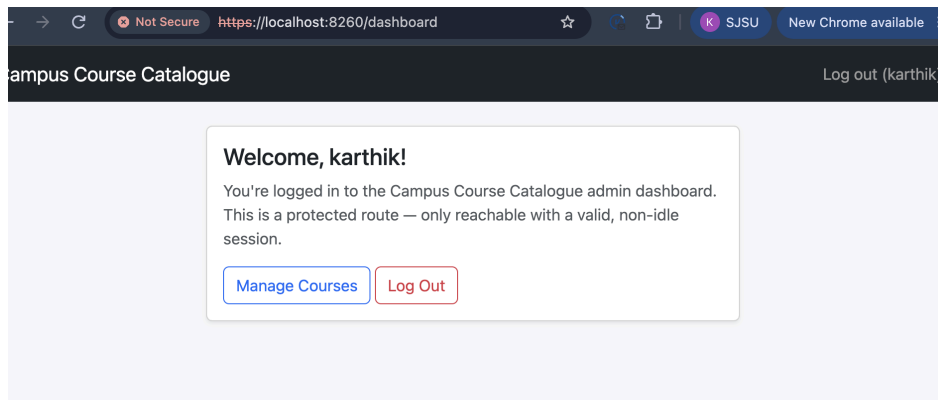
Reached only with a valid, non-idle, non-logged-out session, shows a personalized welcome and a logout link.

For now we have a hardcoded user (username : Karthik , pwsd: data 260)



The screenshot shows a web browser at the URL <https://localhost:8260/login>. The page title is "Campus Course Catalogue". It features a "Log In" form with the following elements:

- Username:** A text input field containing the value "karthik".
- Password:** A password input field with masked characters ".....".
- Log In Button:** A blue button labeled "Log In".
- Back to home Link:** A blue link labeled "< Back to home" located below the form.

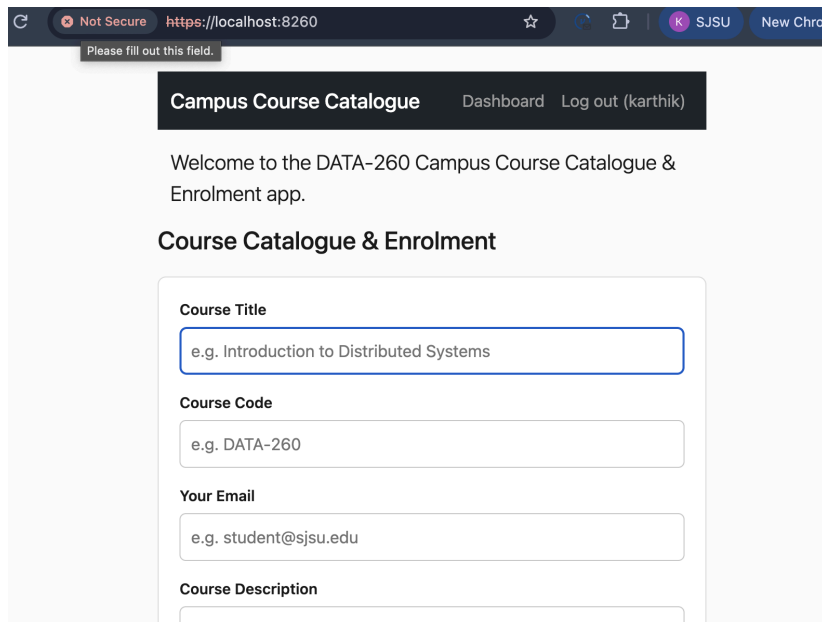


The screenshot shows the dashboard page at the URL <https://localhost:8260/dashboard>. The page title is "Campus Course Catalogue" and the user is logged in as "karthik". The dashboard content includes:

- Welcome, karthik!**
- Message:** "You're logged in to the Campus Course Catalogue admin dashboard. This is a protected route — only reachable with a valid, non-idle session."
- Buttons:** Two buttons are displayed: "Manage Courses" (blue) and "Log Out" (red).

4) Home Page (/) :

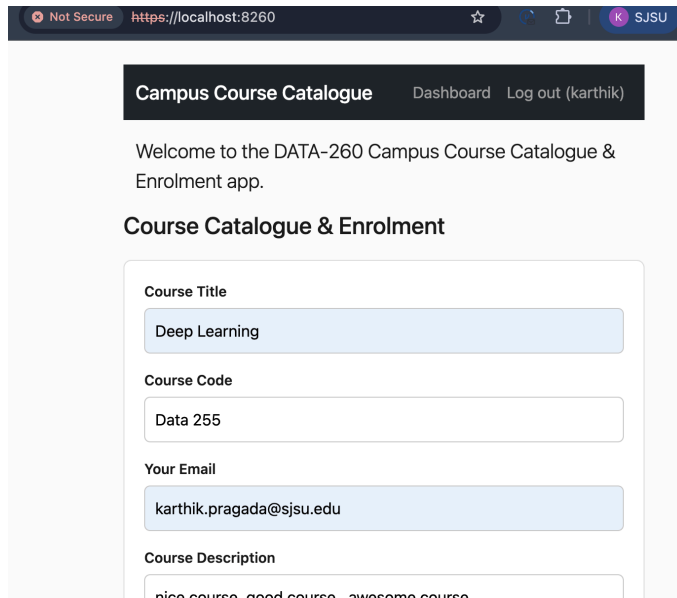
Once clicked on Manage Courses we get routed to the home page where Navbar now shows Dashboard + Log out links instead of Log in.



The screenshot shows a web browser at <https://localhost:8260>. The page has a dark header with "Campus Course Catalogue" and links for "Dashboard" and "Log out (karthik)". Below the header, a welcome message reads: "Welcome to the DATA-260 Campus Course Catalogue & Enrolment app." The main section is titled "Course Catalogue & Enrolment" and contains a form with the following fields:

- Course Title**: A text input field with the placeholder text "e.g. Introduction to Distributed Systems".
- Course Code**: A text input field with the placeholder text "e.g. DATA-260".
- Your Email**: A text input field with the placeholder text "e.g. student@sjsu.edu".
- Course Description**: A text input field with a placeholder text that is partially visible as "e.g. Introduction to Distributed Systems".

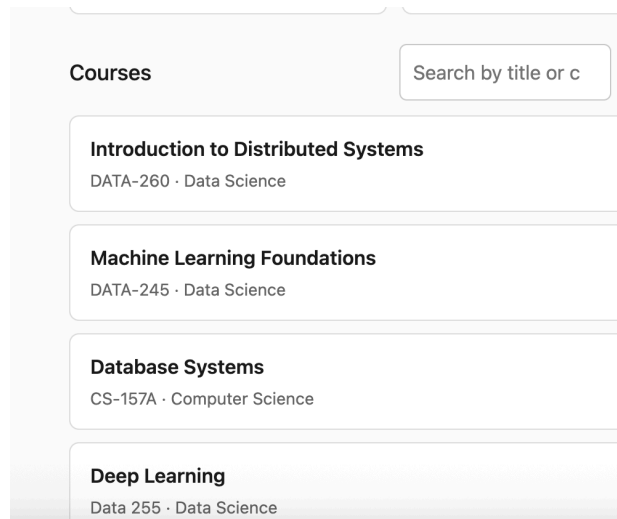
Adding a course named deep learning here to test the form functionality



This screenshot shows the same "Course Catalogue & Enrolment" form, but with test data entered into the fields:

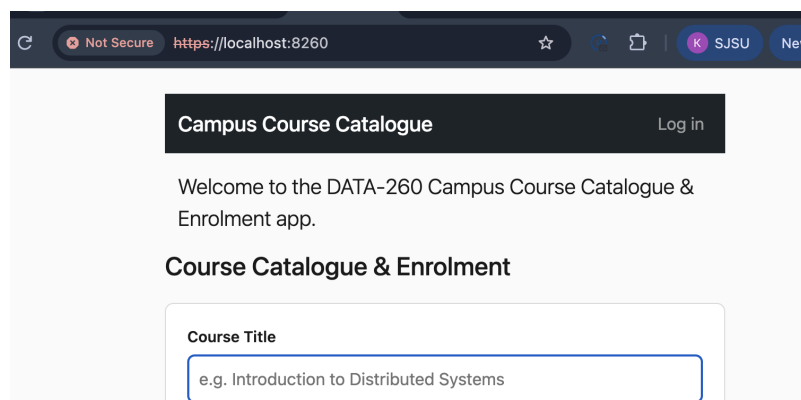
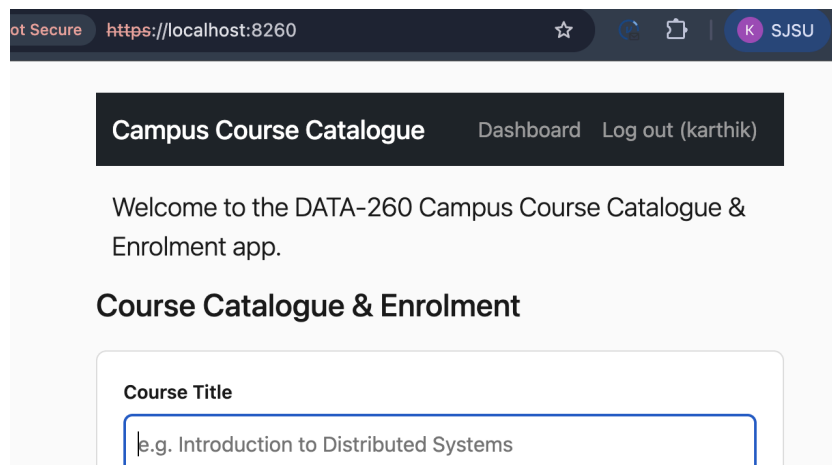
- Course Title**: "Deep Learning"
- Course Code**: "Data 255"
- Your Email**: "karthik.pragada@sjsu.edu"
- Course Description**: "nice course good course awesome course"

Result:



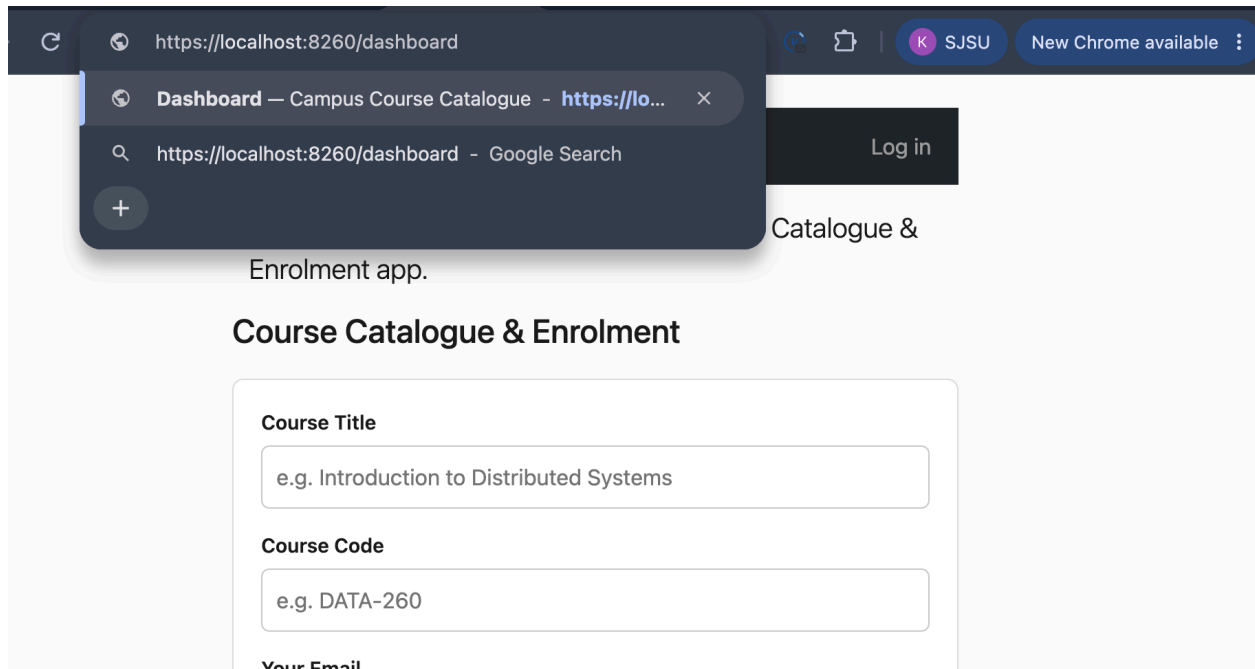
5. Logout

When clicked on Logout we can see the login option again and we have logged out of the account.

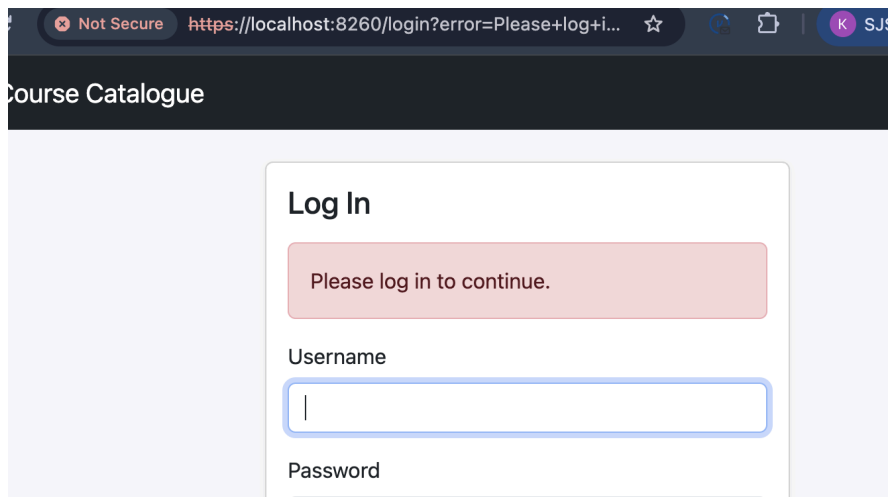


6. Direct “/dashboard” access

while logged out Redirects to /login with a "Please log in to continue. " alert instead of exposing the protected page.



Error message pops up as expected.



Session management

1) Set-Cookie header with all three attributes

The login response sets s8360_session with httponly; samesite=lax; secure

Set-cookie-

The screenshot shows a web browser at `https://localhost:8260/dashboard`. The page displays a welcome message for 'karthik' and buttons for 'Manage Courses' and 'Log Out'. The Network tab is open, showing the 'login' request. The response headers include 'Set-Cookie: s8360_session=eyJ1c2VyIjojImthcnRoawSiCA1bG9naW5fdGltZS16IDE3ODk4ODQwMTEuZmZlMzE2LCA1bGFzdF9zZWVudjogMTc4OTQ4ODAlMS43MTUzMTZ9.aq-GMw.IA-xGtn1MqkJMbv30pbTt5lQw; path=/; Max-Age=1209600; httponly; samesite=lax; secure'.

2) A logged-out session can't be reused

The cookie returned 200 before logout; the same cookie after logout gets 303 to /login.

```
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % curl -sk -c /tmp/c.txt -o /dev/null -X POST https://localhost:8260/login -d "username=karthik&password=data260"
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % curl -sk -b /tmp/c.txt -o /dev/null -w "before logout: HTTP %{http_code}\n" https://localhost:8260/dashboard
before logout: HTTP 200
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % curl -sk -b /tmp/c.txt -o /dev/null https://localhost:8260/logout
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % curl -sk -b /tmp/c.txt -D - -o /dev/null https://localhost:8260/dashboard
HTTP/1.1 303 See Other
date: Sun, 20 Sep 2026 07:10:47 GMT
server: uvicorn
content-length: 0
location: /login?error=Please+log+in+to+continue.
vary: Cookie
set-cookie: s8360_session=null; path=/; expires=Thu, 01 Jan 1970 00:00:00 GMT; httponly; samesite=lax; secure
(base) prags@Karthiks-MacBook-Air HW1_DATA260 %
```

3) Idle timeout (60 seconds)

Immediately after login: immediately: HTTP 200, so the dashboard was reachable.

After sleep 65, same cookie: HTTP/1.1 303 See Other with location:

/login?error=Please+log+in+to+continue. The server refused the idle session and cleared the cookie.

After more than 60 seconds idle, the same cookie is redirected to /login.

```
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % date
Sun Sep 20 01:00:56 PDT 2026
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % curl -sk -c /tmp/i.txt -o /dev/null -X POST https://localhost:8260/login -d "username=karthik&password=data260"
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % curl -sk -b /tmp/i.txt -o /dev/null -w "immediately: HTTP %{http_code}\n" https://localhost:8260/dashboard
immediately: HTTP 200
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % sleep 65
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % curl -sk -b /tmp/i.txt -D - -o /dev/null https://localhost:8260/dashboard
HTTP/1.1 303 See Other
date: Sun, 20 Sep 2026 08:02:49 GMT
server: uvicorn
content-length: 0
location: /login?error=Please+log+in+to+continue.
vary: Cookie
set-cookie: s8360_session=null; path=/; expires=Thu, 01 Jan 1970 00:00:00 GMT; httponly; samesite=lax; secure

(base) prags@Karthiks-MacBook-Air HW1_DATA260 % date
Sun Sep 20 01:02:55 PDT 2026
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % █
```

Code - Auth.py

```
Explorer
  HW1_DATA260
  > __pycache__
  > .venv
  > certs
  > corpus/hw03
  > reports
  > screenshots
  > scripts
  > src
  > static
  > templates
  > .gitignore
  agent_graph.py
  AGENT.md
  agents_demo.py
  auth.py
  {} CORPUS_MANIFEST.json
  Dockerfile
  DOMAIN_SCHEMA.md
  hw1_client.py
  main.py
  ! questions.yaml
  rag_chunking_comparison.py
  README.md
  requirements.txt
  run_nondeterminism.py
  run_schema_experiment.py
  SOURCES.md
  verify_hw01.py
  verify_hw02.py
  verify_hw03.py

  Outline
  Timeline

JS script.js x # style.css x main.py x agent_graph.py x agents_demo.py x run_schema_experiment.py x run_nondeterminism.py x auth.py M x
auth.py
  2 import time
  3
  4 from fastapi import APIRouter, Form, Request
  5 from fastapi.responses import RedirectResponse
  6 from fastapi.templating import Jinja2Templates
  7
  8 router = APIRouter()
  9 templates = Jinja2Templates(directory="templates")
  10
  11 # Single hardcoded demo user -- no real user database for this assignment.
  12 DEMO_USERNAME = "karthik"
  13 DEMO_PASSWORD = "data260"
  14
  15 IDLE_TIMEOUT_SECONDS = 60
  16
  17 _LOGGED_OUT_BEFORE: dict[str, float] = {}
  18
  19
  20 def get_current_user(request: Request) -> str | None:
  21     """Returns the logged-in username, or None if not logged in, the
  22     session has been idle past IDLE_TIMEOUT_SECONDS, or this specific
  23     cookie was issued before the user's most recent logout (a replayed,
  24     already-logged-out cookie). Either failure clears the session
  25     server-side on the spot, so the same cookie can't be reused again."""
  26     user = request.session.get("user")
  27     if not user:
  28         return None
  29     last_seen = request.session.get("last_seen", 0)
  30     login_time = request.session.get("login_time", 0)
  31     if time.time() - last_seen > IDLE_TIMEOUT_SECONDS:
  32         request.session.clear()
  33         return None
  34     if login_time <= _LOGGED_OUT_BEFORE.get(user, 0):
  35         request.session.clear()
  36         return None
  37     request.session["last_seen"] = time.time()
  38     return user
  39
  40
  41 @router.get("/login")
  42 def login_page(request: Request, error: str = ""):
  43     if get_current_user(request):
  44         return RedirectResponse(url="/dashboard", status_code=303)
  45     return templates.TemplateResponse(request, "login.html", {"error": error})
  46
  47
  48 @router.post("/login")
  49 def login_submit(request: Request, username: str = Form(...), password: str = Form(...)):
  50     if username == DEMO_USERNAME and password == DEMO_PASSWORD:
```

Part 2

(Compare Three LlamaIndex Chunking Techniques (Retrieval-Only RAG))

Built a retrieval-only RAG comparison over a 207,022-byte domain corpus (16 real SJSU pages covering course-catalogue programs, policies, and enrollment — DOMAIN_ID=0). Embeddings: sentence-transformers/all-MiniLM-L6-v2 (384-dim), local via llama-index-embeddings-huggingface.

Dataset

The corpus is 16 public SJSU pages for DOMAIN_ID = 0 (Campus Course Catalogue and Enrolment), 207,022 bytes in total, above the 200 KB minimum. It covers degree program requirements, graduate and undergraduate policies, grading, registration and add/drop, the academic calendar and general education requirements, all collected on 2026-09-18. Four pages were downloaded directly from sjsu.edu and cleaned to plain text. The other twelve, from catalog.sjsu.edu, block automated requests, so their text was copied from a browser and some of the longer pages were condensed. The pages come from different catalog years (mostly 2022–23), and one file, msda_curriculum_courses.txt, still contains marketing text.

1) The 16 corpus files.

```
[(base) prags@Karthiks-MacBook-Air HW1_DATA260 % ls -l corpus/hw03
total 456
-rw-r--r--@ 1 prags  staff   2180 Sep 18 00:03 academic_calendar.txt
-rw-r--r--@ 1 prags  staff   5854 Sep 17 23:50 adding_dropping_classes.txt
-rw-r--r--@ 1 prags  staff   7411 Sep 17 23:54 applied_data_intelligence_ms_program.txt
-rw-r--r--@ 1 prags  staff  11944 Sep 17 23:52 course_credit_numbering.txt
-rw-r--r--@ 1 prags  staff   6846 Sep 17 23:51 data_analytics_ms_program.txt
-rw-r--r--@ 1 prags  staff   6611 Sep 17 23:51 data_science_ms_program.txt
-rw-r--r--@ 1 prags  staff   1414 Sep 18 00:02 general_education_overview.txt
-rw-r--r--@ 1 prags  staff  20199 Sep 18 00:04 general_education_requirements_full.txt
-rw-r--r--@ 1 prags  staff  14997 Sep 17 23:53 grades_policy.txt
-rw-r--r--@ 1 prags  staff  26739 Sep 17 23:56 graduate_policies_procedures.txt
-rw-r--r--@ 1 prags  staff  15165 Sep 17 23:59 masters_requirements.txt
-rw-r--r--@ 1 prags  staff  10828 Sep 17 23:49 msda_curriculum_courses.txt
-rw-r--r--@ 1 prags  staff   2258 Sep 17 23:49 msds_admission_requirements.txt
-rw-r--r--@ 1 prags  staff  22939 Sep 17 23:50 msds_faq.txt
-rw-r--r--@ 1 prags  staff  22719 Sep 18 00:01 registration_and_attendance.txt
-rw-r--r--@ 1 prags  staff  28918 Sep 17 23:57 undergraduate_policies_procedures.txt
```

2) Total 207,022 bytes, above the 200 KB minimum

```
[(base) prags@Karthiks-MacBook-Air HW1_DATA260 % cat corpus/hw03/*.txt | wc -c
207022
(base) prags@Karthiks-MacBook-Air HW1_DATA260 %
```

3) Each file's name, byte size and SHA-256 hash.

```
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % head -25 CORPUS_MANIFEST.json
{
  "domain_id": 0,
  "domain": "Campus Course Catalogue and Enrolment",
  "total_bytes": 207022,
  "file_count": 16,
  "files": [
    {
      "filename": "corpus/hw03/academic_calendar.txt",
      "byte_size": 2180,
      "sha256": "e514c8e52700102d980b8eabbe3295adacc26ff0a3acc80ac1ca27afc2ab70c6"
    },
    {
      "filename": "corpus/hw03/adding_dropping_classes.txt",
      "byte_size": 5854,
      "sha256": "fe4027e8cb2a538f858d5bcd4e3935dc52130cb03647a7fa62cfa954c0f23ed5"
    },
    {
      "filename": "corpus/hw03/applied_data_intelligence_ms_program.txt",
      "byte_size": 7411,
      "sha256": "53b4a97826479fc7f0ac1a28dc330a687d4dd7550f686c129afcb8fde2ac8805"
    },
    {
      "filename": "corpus/hw03/course_credit_numbering.txt",
      "byte_size": 11944,
      "sha256": "70e98e95230a380b08e48501c0cd326491d42cec5339afe3f4be4f486ee45b94"
    }
  ]
}
```

4) Source URL and access date (2026-09-18) for every file.

```
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % head -30 SOURCES.md
# SOURCES.md - HW3 Part 2 domain corpus

Domain: DOMAIN_ID=0, Campus Course Catalogue and Enrolment (San José State University).
All 16 documents are real, public SJSU pages relevant to course catalogues, degree/program
requirements, and enrollment policy, fetched live and saved as clean plain text (HTML
chrome and navigation stripped) under `corpus/hw03/`. Access date for all sources below:
**2026-09-18**. Byte sizes and SHA-256 hashes for each file are in `CORPUS_MANIFEST.json`.

| Local file | Source URL |
|---|---|
| `corpus/hw03/data_science_ms_program.txt` | https://catalog.sjsu.edu/preview_program.php?catoid=17&poide=14075 |
| `corpus/hw03/data_analytics_ms_program.txt` | https://catalog.sjsu.edu/preview_program.php?catoid=13&poide=7675 |
| `corpus/hw03/applied_data_intelligence_ms_program.txt` | https://catalog.sjsu.edu/preview_program.php?catoid=17&poide=13784 |
| `corpus/hw03/msds_admission_requirements.txt` | https://www.sjsu.edu/cs/programs/ms-datascience/admission-requirements.php |
| `corpus/hw03/msda_curriculum_courses.txt` | https://sjsu.edu/applied-data-science/msda/curriculum-courses/index.php |
| `corpus/hw03/msds_faq.txt` | https://www.sjsu.edu/cs/programs/ms-datascience/msds-faq.php |
| `corpus/hw03/course_credit_numbering.txt` | https://catalog.sjsu.edu/content.php?catoid=13&navoid=4902 |
| `corpus/hw03/grades_policy.txt` | https://catalog.sjsu.edu/content.php?catoid=15&navoid=5336 |
| `corpus/hw03/graduate_policies_procedures.txt` | https://catalog.sjsu.edu/content.php?catoid=13&navoid=4899 |
| `corpus/hw03/undergraduate_policies_procedures.txt` | https://catalog.sjsu.edu/content.php?catoid=13&navoid=4884 |
| `corpus/hw03/masters_requirements.txt` | https://catalog.sjsu.edu/content.php?catoid=13&navoid=4886 |
| `corpus/hw03/registration_and_attendance.txt` | https://catalog.sjsu.edu/content.php?catoid=13&navoid=4897 |
| `corpus/hw03/adding_dropping_classes.txt` | https://ischool.sjsu.edu/adding-and-dropping-classes |
| `corpus/hw03/general_education_overview.txt` | https://catalog.sjsu.edu/content.php?catoid=10&navoid=661 |
| `corpus/hw03/general_education_requirements_full.txt` | https://catalog.sjsu.edu/content.php?catoid=10&navoid=659 |
| `corpus/hw03/academic_calendar.txt` | https://catalog.sjsu.edu/content.php?catoid=13&navoid=4974 |

## Extraction method

Most pages (`catalog.sjsu.edu`) return an empty response to plain HTTP clients (bot
(base) prags@Karthiks-MacBook-Air HW1_DATA260 %
```

Environment & Model

The pipeline runs on Python 3.12.13 in a local virtual environment on a MacBook Air (Apple M4, 16 GB), with llama-index 0.14.24, llama-index-embeddings-huggingface 0.8.0, sentence-transformers 6.0.1, faiss-cpu 1.15.1, numpy 2.5.3 and pandas 3.0.6. Embeddings come from the public sentence-transformers/all-MiniLM-L6-v2 model, pulled

from HuggingFace and run locally. It produces 384-dimensional vectors. The same model is used for all three chunking techniques and for the semantic splitter's boundary detection, so the only thing that changes between pipelines is the chunking method. The vector store is LlamaIndex's in-memory SimpleVectorStore, so nothing leaves the machine.

1) Installing

```
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % pip install llama-index llama-index-embeddings-huggingface sentence-transformers faiss-cpu numpy pandas
Collecting llama-index
  Using cached llama_index-0.14.24-py3-none-any.whl.metadata (14 kB)
Collecting llama-index-embeddings-huggingface
  Using cached llama_index_embeddings_huggingface-0.8.0-py3-none-any.whl.metadata (459 bytes)
Collecting sentence-transformers
  Downloading sentence_transformers-6.1.0-py3-none-any.whl.metadata (20 kB)
Collecting faiss-cpu
  Using cached faiss_cpu-1.15.1-cp310-abi3-macosx_14_0_arm64.whl.metadata (7.8 kB)
Requirement already satisfied: numpy in /Users/prags/miniconda3/lib/python3.13/site-packages (2.4.4)
Requirement already satisfied: pandas in /Users/prags/miniconda3/lib/python3.13/site-packages (3.0.2)
Collecting llama-index-core<0.15.0,>=0.14.24 (from llama-index)
  Using cached llama_index_core-0.14.24-py3-none-any.whl.metadata (2.6 kB)
Collecting llama-index-embeddings-openai<0.7,>=0.6.0 (from llama-index)
  Using cached llama_index_embeddings_openai-0.6.0-py3-none-any.whl.metadata (401 bytes)
Collecting llama-index-llms-openai<0.8,>=0.7.0 (from llama-index)
  Using cached llama_index_llms_openai-0.7.10-py3-none-any.whl.metadata (3.0 kB)
Collecting nltk<3.9.3 (from llama-index)
  Using cached nltk-3.10.3-py3-none-any.whl.metadata (3.2 kB)
Collecting aiohttp<4,>=3.8.6 (from llama-index-core<0.15.0,>=0.14.24->llama-index)
  Downloading aiohttp-3.14.3-cp313-cp313-macosx_11_0_arm64.whl.metadata (8.3 kB)
Collecting aiosqlite (from llama-index-core<0.15.0,>=0.14.24->llama-index)
  Using cached aiosqlite-0.22.1-py3-none-any.whl.metadata (4.3 kB)
Collecting banks<3,>=2.3.0 (from llama-index-core<0.15.0,>=0.14.24->llama-index)
  Using cached banks-2.5.1-py3-none-any.whl.metadata (12 kB)
Collecting dataclasses-json (from llama-index-core<0.15.0,>=0.14.24->llama-index)
  Using cached dataclasses_json-0.6.7-py3-none-any.whl.metadata (25 kB)
Collecting deprecated<1.2.9.3 (from llama-index-core<0.15.0,>=0.14.24->llama-index)
  Using cached deprecated-1.3.1-py2.py3-none-any.whl.metadata (5.9 kB)
```

2) Installed versions

```
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % pip list | grep -iE "^(llama-index|llama-index-core|llama-index-embeddings-huggingface|sentence-transformers|faiss-cpu|numpy|pandas) "
faiss-cpu              1.15.1
llama-index            0.14.24
llama-index-core       0.14.24
llama-index-embeddings-huggingface 0.8.0
numpy                  2.4.4
pandas                 3.0.2
sentence-transformers  6.1.0
(base) prags@Karthiks-MacBook-Air HW1_DATA260 %
```

3) The embedding model loading

```
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % python -c "from llama_index.embeddings.huggingface import HuggingFaceEmbedding; m = HuggingFaceEmbedding(model_name='sentence-transformers/all-MiniLM-L6-v2'); v = m.get_query_embedding('course catalogue'); print('model: all-MiniLM-L6-v2, embedding dimension:', len(v), ', first 8 values:', [round(x, 4) for x in v[:8]])"
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 103/103 [00:00<00:00, 12551.96it/s]
model: all-MiniLM-L6-v2, embedding dimension: 384 , first 8 values: [-0.0045, -0.0326, -0.0194, 0.0323, -0.0156, -0.0258, -0.0522, -0.0166]
(base) prags@Karthiks-MacBook-Air HW1_DATA260 %
```

Proff that that it was pulled from hugging face

```
(base) prags@Karthiks-MacBook-Air HW1_DATA260 % ls ~/.cache/huggingface/hub | grep -i minilm
models--sentence-transformers--all-MiniLM-L6-v2
(base) prags@Karthiks-MacBook-Air HW1_DATA260 %
```

Retrieval-only function (shared by all three techniques)

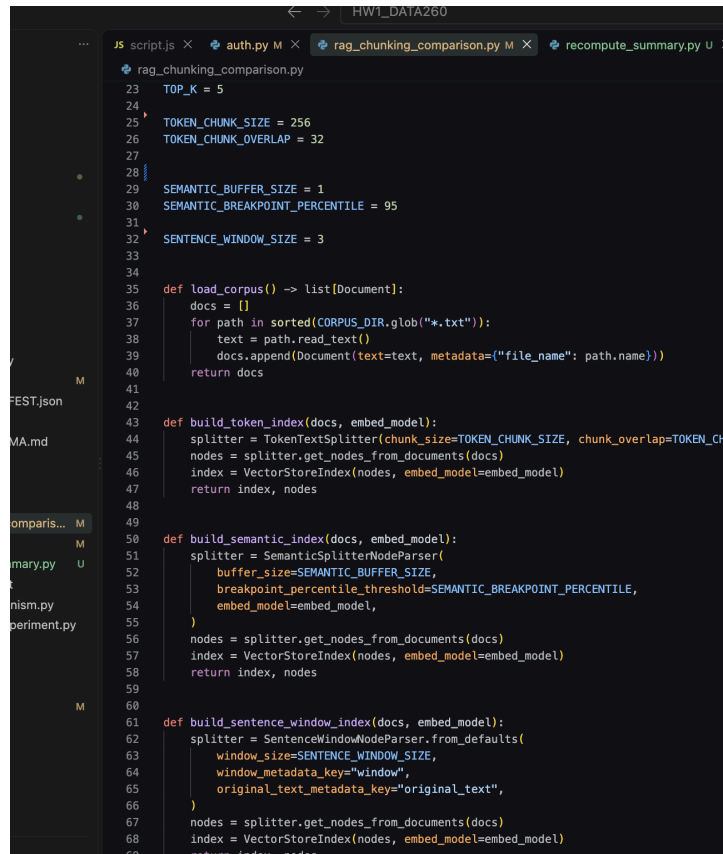
Given an index, a query and k, retrieve_and_analyze (1) embeds the query and reports its dimension and first 8 values; (2) retrieves the top-k nodes and records the retriever's store score; (3) re-embeds each returned chunk's text explicitly and computes the cosine similarity to the query vector; (4) records chunk length and a 160-character preview; (5) prints the shapes of the query vector and the stacked document vectors; and (6) times the similarity search only. Every retrieved row is saved to reports/hw03/raw/ as JSON and CSV.

```
HW1_DATA260
rag_chunking_comparison.py
61 def build_sentence_window_index(docs, embed_model):
62     splitter = SentenceWindowNodeParser.from_defaults(
63         window_metadata_key="window",
64         original_text_metadata_key="original_text",
65     )
66
67     nodes = splitter.get_nodes_from_documents(docs)
68     index = VectorStoreIndex(nodes, embed_model=embed_model)
69     return index, nodes
70
71
72 def cosine_sim(a: np.ndarray, b: np.ndarray) -> float:
73     return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))
74
75
76 def retrieve_and_analyze(index, embed_model, technique: str, question_id: str, query: str, k: int = TOP_K, verbose: bool = True):
77     query_vec = np.array(embed_model.get_query_embedding(query))
78
79     retriever = index.as_retriever(similarity_top_k=k)
80     start = time.perf_counter()
81     retrieved = retriever.retrieve(query)
82     latency_ms = (time.perf_counter() - start) * 1000
83
84     doc_vecs = []
85     rows = []
86     for rank, node_with_score in enumerate(retrieved, start=1):
87         node = node_with_score.node
88         chunk_text = node.get_content()
89         doc_vec = np.array(embed_model.get_text_embedding(chunk_text))
90         doc_vecs.append(doc_vec)
91         sim = cosine_sim(query_vec, doc_vec)
92         rows.append({
93             "rank": rank,
94             "store_score": round(float(node_with_score.score), 6) if node_with_score.score is not None else None,
95             "cosine_sim": round(sim, 6),
96             "chunk_len": len(chunk_text),
97             "source_file": node.metadata.get("file_name"),
98             "preview": chunk_text[:160].replace("\n", " ")
99         })
100
101     doc_matrix = np.stack(doc_vecs) if doc_vecs else np.zeros((0, len(query_vec)))
102
103     if verbose:
104         print(f"\n=== [{technique}] {question_id: {query.strip()[:80]} ===")
105         print(f"query embedding dim: {query_vec.shape[0]}, first 8 values: {query_vec[:8].round(4).tolist()}")
106         print(f"query vector shape: {query_vec.shape}, stacked doc vectors shape: {doc_matrix.shape}")
107         print(f"{'rank':<5}{'store_score':<13}{'cosine_sim':<12}{'chunk_len':<10}{'source_file':<45}preview")
108         for r in rows:
109             print(f"{'rank':<5}{str(r['store_score']):<13}{r['cosine_sim']:<12}{r['chunk_len']:<10}{str(r['source_file']):<45}{r['preview']}")
110         print(f"retrieval latency: {latency_ms:.2f} ms")
```

Token chunking

1. Chunking

Splits the text every 256 tokens with a 32-token (12.5%) overlap. The size keeps each chunk on roughly one policy or requirement, and the overlap means a fact that straddles a boundary still appears whole in at least one chunk. It ignores meaning: boundaries fall wherever the token count lands.



```
HW1_DATA260
rag_chunking_comparison.py
23 TOP_K = 5
24
25 TOKEN_CHUNK_SIZE = 256
26 TOKEN_CHUNK_OVERLAP = 32
27
28
29 SEMANTIC_BUFFER_SIZE = 1
30 SEMANTIC_BREAKPOINT_PERCENTILE = 95
31
32 SENTENCE_WINDOW_SIZE = 3
33
34
35 def load_corpus() -> list[Document]:
36     docs = []
37     for path in sorted(CORPUS_DIR.glob("*.txt")):
38         text = path.read_text()
39         docs.append(Document(text=text, metadata={"file_name": path.name}))
40     return docs
41
42
43 def build_token_index(docs, embed_model):
44     splitter = TokenTextSplitter(chunk_size=TOKEN_CHUNK_SIZE, chunk_overlap=TOKEN_CHUNK_OVERLAP)
45     nodes = splitter.get_nodes_from_documents(docs)
46     index = VectorStoreIndex(nodes, embed_model=embed_model)
47     return index, nodes
48
49
50 def build_semantic_index(docs, embed_model):
51     splitter = SemanticSplitterNodeParser(
52         buffer_size=SEMANTIC_BUFFER_SIZE,
53         breakpoint_percentile_threshold=SEMANTIC_BREAKPOINT_PERCENTILE,
54         embed_model=embed_model,
55     )
56     nodes = splitter.get_nodes_from_documents(docs)
57     index = VectorStoreIndex(nodes, embed_model=embed_model)
58     return index, nodes
59
60
61 def build_sentence_window_index(docs, embed_model):
62     splitter = SentenceWindowNodeParser.from_defaults(
63         window_size=SENTENCE_WINDOW_SIZE,
64         window_metadata_key="window",
65         original_text_metadata_key="original_text",
66     )
67     nodes = splitter.get_nodes_from_documents(docs)
68     index = VectorStoreIndex(nodes, embed_model=embed_model)
69     return index, nodes
```

Chunks	Avg length	Min	Median	Max
204	1158.2 chars	174	1218	1530

A real chunk from this technique: the first one containing the \$200.00 late-fee fact (from registration_and_attendance.txt , 1275 characters, shortened here):

2. Indexing (in memory)

The nodes are loaded into a LlamaIndex VectorStoreIndex with no external store configured, so LlamaIndex uses its default in-memory SimpleVectorStore. Nothing is persisted or sent anywhere. This is the last line of the builder above: `index = VectorStoreIndex(nodes, embed_model=embed_model)`.

3. Retrieval-only output for the shared question (q3)

The retrieval function (documented once above) runs on this technique's index. The table lists rank, store score, recomputed cosine, chunk length, source file and a 160-character preview.

```
(base) prags@Karthis-MacBook-Air HW1_DATA260 % python - <<'EOF'
import yaml, rag_chunking_comparison as m
from llm import embeddings,huggingface_import HuggingFaceEmbedding
embed = HuggingFaceEmbedding(model_name=EMBED_MODEL_NAME)
q = {x['id']: x["question"] for x in yaml.safe_load(open("questions.yaml"))["questions"]}
index_nodes = m.build_token_index(m.load_corpus(), embed)
m.retrieve_and_analyze(index, embed, "token", "q3", q["q3"], k=5)
EOF
Warning: You are sending unauthorized requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% [████████████████████] 103/103 [00:00<00:00, 10750.3ait/s]

=== [token] q3: What are the two specialization tracks in SJSU's MS in Applied Data Intelligence ===
query embedding dim: 386, first 6 values: [-0.0274, -0.0687, -0.022, 0.0218, 0.0002, -0.0398, -0.0296, 0.0243]
query vector shape: (386,) , stacked doc vectors shape: (5, 384)
rank score cosine_sim chunk_len source_file
1 0.675474 0.636141 1326 msda_curriculum_courses.txt
    preview
use, and evaluate applied data intelligence technologies and tools. Apply Necessary Skills Apply effective oral and writ
ten communication skills necessary to pr
2 0.667293 0.644114 1434 applied_data_intelligence_ms_program.txt Program: Applied Data Intelligence, MS - San José State University Source: https://catalog.sjsu.edu/preview_program.php?catoid=17&poid=13784 Applied Data Intell
3 0.666494 0.653941 1362 data_science_ms_program.txt Program: Data Science, MS - San José State University Source: https://catalog.sjsu.edu/preview_program.php?catoid=17&poid=1480 Applied Data Science, MS Admission Re
4 0.649618 0.636805 1095 msda_curriculum_courses.txt Program Frequently Asked Questions Want to learn more about our MS in Applied Data Intelligence degree? Explore our A
page to get detailed information about
5 0.656676 0.622582 1393 msda_curriculum_courses.txt data into actionable insights, and step confidently into advanced roles across technology, business and research. Apply
Now Request Information Why Earn Your Ap
retrieval latency: 7.93 ms
(base) prags@Karthis-MacBook-Air HW1_DATA260 % █
```

Semantic chunking

1. Chunking

Embeds each sentence (buffer_size=1) with the same MiniLM model and starts a new chunk only where the embedding distance to the next sentence is in the top 5% (breakpoint_percentile_threshold=95). Chunks follow topic changes, so they are few and large, and their size varies with the text.

```

rag_chunking_comparison.py
35 def load_corpus() -> list[Document]:
36     docs = []
37     for path in sorted(CORPUS_DIR.glob("*.txt")):
38         text = path.read_text()
39         docs.append(Document(text=text, metadata={"file_name": path.name}))
40     return docs
41
42
43 def build_token_index(docs, embed_model):
44     splitter = TokenTextSplitter(chunk_size=TOKEN_CHUNK_SIZE, chunk_overlap=TOKEN_CHUNK_SIZE)
45     nodes = splitter.get_nodes_from_documents(docs)
46     index = VectorStoreIndex(nodes, embed_model=embed_model)
47     return index, nodes
48
49
50 def build_semantic_index(docs, embed_model):
51     splitter = SemanticSplitterNodeParser(
52         buffer_size=SEMANTIC_BUFFER_SIZE,
53         breakpoint_percentile_threshold=SEMANTIC_BREAKPOINT_PERCENTILE,
54         embed_model=embed_model,
55     )
56     nodes = splitter.get_nodes_from_documents(docs)
57     index = VectorStoreIndex(nodes, embed_model=embed_model)
58     return index, nodes
59
60
61 def build_sentence_window_index(docs, embed_model):
62     splitter = SentenceWindowNodeParser.from_defaults(
63         window_size=SENTENCE_WINDOW_SIZE,
64         window_metadata_key="window",
65         original_text_metadata_key="original_text",
66     )
67     nodes = splitter.get_nodes_from_documents(docs)
68     index = VectorStoreIndex(nodes, embed_model=embed_model)
69     return index, nodes
70

```

Chunks	Avg length	Min	Median	Max
85	2434.5 chars	32	2133	7851

Select the correct term. Choose the class you wish to add.

Two ways to select a class: if you know the Class Number, type the 5 digit Class Number and click Enter; or click the green "Search Button" (make sure the Course Career is set to Graduate or Undergraduate) and click "Select Class" to place it in your shopping cart.

Once you have chosen a class, a "Class Enrollment Options" page will appear. Verify the course, enter permission number if required, select any related component classes, and click okay. The Add Classes page will appear showing your classes in your shopping cart. You must click "Finish Enrolling" to actually add the class.

Component Classes: if the course has related components attached (e.g., lab, seminar), the system will prompt you to select the one you wish to add.

Department or Instructor Consent: classes designated "Department Consent Required" require a Perm ...

2. Indexing (in memory)

The nodes are loaded into a LlamaIndex VectorStoreIndex with no external store configured, so LlamaIndex uses its default in-memory SimpleVectorStore. Nothing is persisted or sent anywhere. This is the last line of the builder above: `index = VectorStoreIndex(nodes, embed_model=embed_model)`.

3. Retrieval-only output for the shared question (q3)

The retrieval function (documented once above) run on this technique's index. The table lists rank, store score, recomputed cosine, chunk length, source file and a 160-character preview.

```
(base) prags@Karthis-MacBook-Air HW1_DATA260 % python - <<'EOF'
import yaml, rag_chunking_comparison as m
from llama_index.embeddings.Huggingface import HuggingFaceEmbedding
embed = HuggingFaceEmbedding(model_name='BMBE_MODEL_NAME')
q = x['id']:x['question'] for x in yaml.safe_load_all('questions.yaml'))['questions']
index_nodes = m.build_semantic_index(m.load_corpus(), embed)
retrieve_and_analyze(index, embed, "semantic", "q3", q["q3"], k=5)
EOF
Warning: You are sending unauthorized requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% [██████████] 103/103 [00:00<00:00, 10392.12it/s]

=== semantic q3: What are the two specialization tracks in JSJU's MS in Applied Data Intelligence ===
query embedding dim: 384, first 8 values: [-0.0274, -0.0687, -0.022, 0.0218, 0.0082, -0.0398, -0.0296, 0.0243]
token shape: (384,) , stacked doc vectors shape: (5, 384)
rank score cosine sim chunk_len source_file:
1. 0.608653 0.6847 3777 msds_curriculum_courses.txt
2. 0.607953 0.648885 5423 applied_data_intelligence_ms_program.txt
3. 0.659187 0.655845 1249 data_science_ms_program.txt
4. 0.652115 0.637578 5621 ms_admissions_research_projects.txt
5. 0.656212 0.67589 3421 msds_curriculum_courses.txt
MS in Data Analytics - have accomplished.
1. 0.626916 0.630787 239 data_analytics_ms_program.txt
2. 0.607075 Data Analytics, MS Admissions
retrieval latency: 6.79 ms
(base) prags@Karthis-MacBook-Air HW1_DATA260 %
```

Sentence-window chunking

1. Chunking

Makes every sentence its own node and stores the three sentences on each side in the node's `window` metadata (window_size=3). Only the single sentence is embedded and searched; the window is kept so surrounding context could be supplied later. Nodes are very small and numerous.

```

35 def load_corpus() -> list[Document]:
36     docs = []
37     for path in sorted(CORPUS_DIR.glob("*.txt")):
38         text = path.read_text()
39         docs.append(Document(text=text, metadata={"file_name": path.name}))
40     return docs
41
42
43 def build_token_index(docs, embed_model):
44     splitter = TokenTextSplitter(chunk_size=TOKEN_CHUNK_SIZE, chunk_overlap=0)
45     nodes = splitter.get_nodes_from_documents(docs)
46     index = VectorStoreIndex(nodes, embed_model=embed_model)
47     return index, nodes
48
49
50 def build_semantic_index(docs, embed_model):
51     splitter = SemanticSplitterNodeParser(
52         buffer_size=SEMANTIC_BUFFER_SIZE,
53         breakpoint_percentile_threshold=SEMANTIC_BREAKPOINT_PERCENTILE,
54         embed_model=embed_model,
55     )
56     nodes = splitter.get_nodes_from_documents(docs)
57     index = VectorStoreIndex(nodes, embed_model=embed_model)
58     return index, nodes
59
60
61 def build_sentence_window_index(docs, embed_model):
62     splitter = SentenceWindowNodeParser.from_defaults(
63         window_size=SENTENCE_WINDOW_SIZE,
64         window_metadata_key="window",
65         original_text_metadata_key="original_text",
66     )
67     nodes = splitter.get_nodes_from_documents(docs)
68     index = VectorStoreIndex(nodes, embed_model=embed_model)
69     return index, nodes

```

Chunks	Avg length	Min	Median	Max
1307	158.3 chars	3	124	4408

Students registering for classes after census are assessed a late registration fee of \$200.00, which begins after census for any classes added (includes Retroactive Adds, which begin the first day after instruction ends).

Adding Classes after the Last Day to Add: before the Census date, students can submit the Pre-census Late Enrollment petition form to the Registrar's office (valid through census date per University Policy F09-2). After census date, students can petition for a late add using the Post-Census Late Enrollment form.

Students registering for classes after census are assessed a late registration fee of \$200.00, which begins after census for any classes added (includes Retroactive Adds, which begin the first day after instruction ends). Students in the Step to College programs are not required to pay the late registration fee.

SJSU has "waitlisting" for all classes. Waitlisting puts you in line to get a space when a class is full (closed class), should a space become available.

The nodes are loaded into a LlamaIndex VectorStoreIndex with no external store configured, so LlamaIndex uses its default in-memory SimpleVectorStore. Nothing is persisted or sent anywhere. This is the last line of the builder above: `index = VectorStoreIndex(nodes, embed_model=embed_model)`.

The retrieval function (documented once above) run on this technique's index. The table lists rank, store score, recomputed cosine, chunk length, source file and a 160-character preview.

```
(base) prags@Karthiks-MacBook-Air: HW1_DATA260 % python -c <<'EOF'
import yaml, rag_chunking_comparison as m
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
embed = HuggingFaceEmbedding(model_name="BGEED_VQOCL_NAME")
q = {x["id"]: x["question"]} for x in yaml.safe_load(open('questions.yaml'))["questions"]
index, nodes = m.build_sentence_window_index(m.load_corpus(c), embed)
m.retrieve_and_analyze(index, embed, "sentence_window", "q3", q["q3"], k=5)
EOF
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% | 183/183 [00:00<00:00, 11496.751it/s]

=== [sentence_window] q3: What are the two specialization tracks in JSJSU's MS in Applied Data Intelligence ===
query embedding dim: 384, first 8 values: [-0.8274, -0.8687, -0.022, 0.0218, 0.0082, -0.8398, -0.0296, 0.0243]
query vector shape: (384,3), stacked doc vectors shape: (5, 384)
link store source cosine sim chunk-link source file
1 0.724426 0.64512 161 msds_curriculum_courses.txt
MS in Data Analytics - have accomplished.
1 0.724426 0.64512 154 msds_curriculum_courses.txt
preparing you to drive innovation.
3 0.712977 0.695679 124 msds_curriculum_courses.txt
ree?
4 0.784737 0.689269 146 msds_curriculum_courses.txt
real-world change happen.
5 0.698282 0.707374 551 applied_data_intelligence_ms_program.txt
Pstoid=17xpid=1378x Applied Data Intel
retrieval latency: 16.87 ms
(base) prags@Karthiks-MacBook-Air: HW1_DATA260 %
```

Comparison

Retrieval quality (5 questions, k = 5)

Technique	Chunks	Avg chunk length	Top-1 cosine	Mean@k cosine	Recall@k	Mean retrieval latency (ms)
Token	204	1158.2 chars	0.7004	0.6412	0.80	7.91
Semantic	85	2434.5 chars	0.666	0.6011	0.80	5.25
Sentence-window	1307	158.3 chars	0.7646	0.6883	0.80	15.83

Top-1 cosine is the highest cosine among the top-5 chunks, averaged over the questions. Recall@k is file-level: a hit means the expected source file appears anywhere in the top 5. Latency covers the similarity search only and varies by machine.

Answer-level check (was the answer text itself retrieved?)

Recall@k only checks the file, so this supplementary check (scripts/answer_level_check.py) asks whether any top-5 chunk contains the expected answer text. Each cell is the rank of the first chunk that does; "-" means none did.

Technique	q1	q2	q3	q4	q5	Answered in top 5
Token	1	3	-	1	1	4/5
Semantic	2	3	-	-	1	3/5
Sentence-window	-	-	-	1	1	2/5

q1 requires two marker strings in one chunk, which disadvantages single-sentence nodes; q2, q4 and q5 use a single marker.

Observations -

Sentence-window has the highest cosine scores (top-1 0.7646, mean@k 0.6883) but retrieved a chunk containing the answer for only 2 of 5 questions, against 4 for Token and 3 for Semantic. High similarity did not mean a useful chunk: single-sentence nodes carry little context, so shared proper nouns dominate the match, and near-identical boilerplate repeated across the three program pages (for example the GVAR sentence) outranked the sentence naming DATA 270 in q2. Semantic chunks are the largest (2434.5 characters on average), which likely dilutes a single fact such as the \$200.00 fee in q4 inside a long registration page.

Recall@k is 0.80 for all three because of q1: the 3.0 GPA rule appears in three documents, and every technique retrieved graduate_policies_procedures.txt, a valid source that the single

expected-file label counted as a miss. q3 failed for every technique. With only five questions these differences are indicative, not conclusive.

Conclusion

Token chunking (256 tokens, 32 overlap) is the best choice for this corpus. It retrieved a chunk containing the answer for 4 of 5 questions, against 3 for Semantic and 2 for Sentence-window, even though Sentence-window had the highest cosine scores, so cosine alone was misleading here. Its mean latency of 7.91 ms is well below Sentence-window's 15.83 ms and close to Semantic's 5.25 ms. The result rests on five questions, and q3 failed for all three techniques.